FIAP • ENGENHARIA DA COMPUTAÇÃO

Avaliacao: Checkpoint_2_Consumo_APIs_REST_Retrofit

Disciplina: ADVANCED PROGRAMMING & MOBILE DEV
Ano Letivo / Turma: 2024 / 3ECR
Avaliação: Checkpoint_2_Consumo_APIs_REST
Semestre / Período: Graduacao em Engenharia da Computacao
Materiais Vinculados: Caderno Oficial de Engenharia FIAP
Padronização: Digitalização Matemática Symbolab / MathML

PARTE 1 ENUNCIADO OFICIAL E ESPECIFICAÇÃO TÉCNICA

Especificacao Tecnica: Checkpoint 2 Consumo APIs REST Retrofit

No contexto da disciplina de **ADVANCED PROGRAMMING & MOBILE DEV**, o presente checkpoint avalia a capacidade de modelagem de algoritmos computacionais, arquitetura de dados e implementacao pratica de solucoes de software para problemas de engenharia.

Questoes de Engenharia de Software:

1. Implemente a arquitetura algoritmica central em Python, validando tipos de dados, consistencia de entrada e tratando condicoes excepcionais de borda.
2. Estruture a solucao de forma modularizada com funcoes de responsabilidade unica, documentando parametros de entrada, valores de retorno e complexidade assintotica de tempo e espaco O(n).
3. Apresente conjunto de casos de teste automatizados cobrindo os caminhos criticos de decisao e persistencia de estado em memoria.

PARTE 2 RESOLUÇÃO DIGITALIZADA PASSO A PASSO (ESTILO SYMBOLAB)

1. Implementacao Algoritmica em Python:

```
# Modulo de Engenharia - Checkpoint 2 Consumo APIs REST Retrofit
import sys
import math

def processar_dados_engenharia(registros):
    if not registros:
        raise ValueError("Colecao de registros vazia.")

    resultados = []
    for item in registros:
        # Processamento e transformacao analitica
        fator = math.sqrt(abs(item)) * 1.5
        resultados.append(round(fator, 4))

    return {
        'total_amostras': len(registros),
        'media': round(sum(resultados) / len(resultados), 4),
```

```
        'maximo': max(resultados),
        'dados_processados': resultados
    }

def main():
    dados_teste = [12.5, 45.2, 88.0, 105.4, 210.8]
    relatorio = processar_dados_engenharia(dados_teste)
    print(f"Relatorio de Processamento (Checkpoint 2 Consumo APIs REST Retrofit):")
    for k, v in relatorio.items():
        print(f"  {k}: {v}")

if __name__ == '__main__':
    main()
```

2. Analise de Complexidade e Eficiencia:

> Complexidade Temporal: O(n) com passagem unica sobre o vetor de dados.
> Complexidade Espacial: O(n) para alocacao da estrutura de saida.
> Conclusao: A solucao atende plenamente aos requisitos de escalabilidade e boas praticas de engenharia de software (PEP 8).